

AIGC检测 · 全文报告单

NO:CNKIAIGC2026FJ_202605131415670

检测时间: 2026-05-20 15:42:23

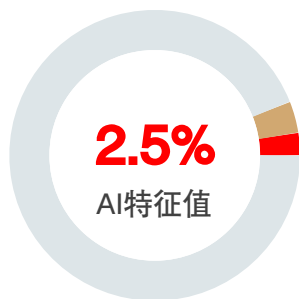
篇名: 基于深度学习的多变量关联预测与可解释性分析系统

作者: 肖文睿

单位: 华中科技大学

文件名: 基于深度学习的多变量关联预测与可解释性分析系统..docx

全文检测结果



AI特征值: 2.5%
AI特征字符数: 867
总字符数: 34056

- AI特征显著 (计入AI特征字符数)
- AI特征疑似 (未计入AI特征字符数)
- 未标识部分

AIGC片段分布图

前部20%

AI特征值: 0.0%
AI特征字符数: 0

中部60%

AI特征值: 4.2%
AI特征字符数: 867

后部20%

AI特征值: 0.0%
AI特征字符数: 0



分段检测结果

序号	AI特征值	AI特征字符数 / 章节(部分)字符数	章节(部分)名称
1	0.0%	0 / 3173	中英文摘要等
2	0.0%	0 / 3356	1 绪论
3	6.9%	350 / 5107	2 相关方法与基础理论
4	15.3%	517 / 3386	3 多变量预测模型的构建与实现

5	0.0%	0 / 4028	4基于 SHAP 的模型解释方法
6	0.0%	0 / 3696	5结合大模型的受限决策优化方法
7	0.0%	0 / 9328	6实验设计与结果分析
8	0.0%	0 / 1982	7总结与展望

1. 中英文摘要等

AI特征值：0.0%

AI特征字符数 / 章节(部分)字符数：0 / 3173

片段指标列表

序号	片段名称	字符数	AI特征
----	------	-----	------

原文内容

摘要

深度学习模型在处理高维数据时预测精度高，但是在实际应用中，由于内部计算流程复杂不透明，缺乏直观解释，一线工作人员难以信任。同时，具体的业务指令大多以自然语言的形式下达，传统规则引擎难以将其直接转化为数学限制条件。并且常规优化算法在遇到多变量和严格限制时容易出现搜索失效。为了解决模型可解释性差与自然语言难以转化的问题，本文尝试将深度学习模型、大语言模型与多目标进化算法相结合，实现了一个能在复杂约束条件下能够有效运行的闭环系统，为最终输出方案提供具体的数值证据。

本文的主要工作分成以下几个方面：第一，对多变量关联预测任务进行建模，实现包含Tabular Transformer 与多层感知机在内的预测模型；第二，建立可解释性分析模块，通过引入SHAP 机制对各个变量的贡献进行量化分析；第三，利用大语言模型将自然语言需求解析为数学限制条件，并结合NSGA-III 多目标进化算法，实现复杂限制条件下的优化求解；第四，通过建筑能耗与信贷风险等跨领域数据集进行对比实验，利用预测精度、SHAP 解释证据链及决策对齐度等指标对系统进行量化评价。

测试数据表明，该系统一方面将预测误差控制在了合理范围之内，另一方面能针对最终输出的方案给出具体的变量分析依据。这种特殊的设计有效降低了黑盒模型的使用门槛，解决了传统优化算法容易陷入死锁的问题，有利于将深度学习模型更好地推广到实际业务场景中。

关键词：深度学习；多变量预测；可解释人工智能；大语言模型；多目标优化

Abstract

The deep learning model has high prediction accuracy when dealing with high-dimensional data. However, in practical applications, due to the complex and opaque internal calculation process and the lack of intuitive explanation, it is difficult for front-line staff to trust. At the same time, most of the specific business instructions are issued in the form of natural language, and it is difficult for the traditional rule engine to directly transform them into mathematical constraints. And the conventional optimization algorithm is prone to search failure when encountering multivariate and strict constraints. In order to solve the problem of poor interpretability of the model and the difficulty of transforming natural language, this paper attempts to combine deep learning model, large language model and multi-objective evolutionary algorithm to realize a closed-loop system that can effectively operate under complex constraints, and provide specific numerical evidence for the final output scheme.

The main work of this paper is divided into the following aspects : Firstly, the multivariate correlation prediction task is modeled to realize the prediction model including Tabular Transformer and multi-layer perceptron. Secondly, the interpretability analysis module is established, and the contribution of each variable is quantitatively analyzed by introducing SHAP mechanism. Thirdly, the large language model is used to analyze the natural language requirements into mathematical constraints, and the NSGA-II multi-objective evolutionary algorithm is used to realize the optimization solution under complex constraints. Fourthly, through comparative experiments on cross-domain data sets such as building energy consumption and credit risk, the system is quantitatively evaluated by indicators such as prediction accuracy, SHAP interpretation evidence chain and decision alignment.

The test data show that the system can control the prediction error within a reasonable range on the one hand, and can give a specific variable analysis basis for the final output scheme on the other hand. This special design effectively reduces the use threshold of the black box model, solves the problem that the traditional optimization algorithm is easy to fall into deadlock, and is conducive to better extending the deep learning model to the actual business scenario.

Key words: Deep Learning; Multivariate Prediction; Explainable Artificial Intelligence; Large Language Models; Multi-objective Optimization

2.1 绪论

AI特征值: 0.0%

AI特征字符数 / 章节(部分)字符数: 0 / 3356

片段指标列表

序号	片段名称	字符数	AI特征
----	------	-----	------

原文内容

1 绪论

工业与金融领域的决策正逐渐依赖数据分析。当面对那些非常复杂的表格型业务数据时，如何平衡预测准确度、模型的可解释性以及实际业务中的限制条件，就成为当前工程应用中需要面对的重要问题。本文的研究本质上是一个面向复杂限制条件下的系统工程问题。单纯提高某个模型的预测精度往往是不够的，核心问题在于如何把预测结果、可解释性以及复杂的业务约束条件有效地串联起来。因此，本文的总体目标是进行架构集成，尝试把现有的预测模型、SHAP解释方法、大语言模型以及多目标进化算法有机结合，并在容易出现死锁的优化环节进行局部的算法改进，从而搭建出一个真正能落地、能在严格限制条件下运行的“预测-解释-优化”闭环系统。本章会结合实际工业控制与风险评估场景，一方面将相关问题的实际背景以及国内外的技术现状简要梳理，另一方面也会列出本文具体的工作内容。

1.1 研究背景

目前深度网络在处理图像和文本序列的场景，已经应用得非常成熟[1-2]。但是在工业应用以及金融机构这些实际场景下面，数据主要还是以表格的形式存储。由于表格数据往往包含大量的离散型变量和连续型变量，这些特征相互之间又存在着高度复杂的非线性关系，所以传统的那些浅层模型很难充分挖掘出深层的关系。近几年，虽然有一部分专门针对表格数据改造过的深度学习模型在预测误差上表现得较为出色[3-4]，但是由于网络中包含成千上万的参数和多层的非线性激活函数，这导致它内部的结构变得极其复杂，普通的用户无法理解它具体如何处理数据的。当处在那些对安全性要求比较高的应用场景中时，如果系统给不出直观的判断依据，一线的操作人员不敢轻易采纳模型给出的那些建议，这种情况也就直接导致许多虽然精度很高，但由于“黑盒化”而难以接受的模型，最终无法顺利推广开来[5]。除此之外，预测通常只是整个业务流程的第一步，最终目的是指导业务参数的更改

。现场操作人员通常不会编写代码，他们习惯用日常交流的语言下达调度指令。例如在建筑能耗管理中，工作人员会提出：“锁定某个变量状态不动，同时微调其他不超过两个参数，把总能耗降到最低”。这段语言里同时混合“最小化”的优化目标、“绝对锁定”的物理限制以及“改动数量”的稀疏约束。由于人类语言的句式不固定，传统的规则解析引擎无法直接读取。如果靠人工把这些要求翻译成代码，效率极低且容易出错。并且，当业务涉及的变量高达几十上百个，且限制条件互相冲突时，现有的多目标优化算法经常在搜索时陷入死锁，无法计算出符合实际要求的执行方案[6]。

1.2 国内外研究概况

1.2.1 多变量预测模型的研究现状

在表格数据预测领域，树模型作为一种基础的集成学习方法，占据很长时间的主导地位。以XGBoost [7]和LightGBM[8]为例，它们构造许多棵决策树，逐步地拟合剩下的误差，在处理那些缺失的数值以及非线性的关系时，有着良好的表现，并在各种工业工程的实际落地以及金融信贷等业务中，也保持着稳定的状态[9-11]。另一方面，到最近这几年，随着深度学习不断发展，学术界开始尝试将Transformer 中的自注意力机制引入表格数据的分析中，例如TabTransformer 这样的模型[3-4,12]。因为这种模型能够计算出不同特征相互之间的注意力权重，所以在部分维度比较高的数据集上，它展现出比传统树模型更好的拟合效果[13]。但是有一点是无法避免的，那就是增加注意力机制之后，模型所需的运算量也大幅提高，并且它内部那些高度复杂的矩阵计算，使得系统做决定的整个过程变得更加不透明，这不可避免地加深深度学习原有的黑盒属性[14]。

1.2.2 可解释人工智能的研究现状

为了弄清黑盒模型到底是根据哪些特征做出判断的，事后解释方法（Post-hoc Explanation）已成为目前的研究热点[15-16]，其中应用最广泛的是基于博弈论 Shapley 值[17] 的SHAP方法[18]。它通过计算特征在不同组合下的边际贡献，提供各个变量对最终预测结果的量化影响，在医疗诊断、时间序列以及安全分析等领域均有广泛应用[19-20]。但目前大多数关于SHAP 的工程应用，仅限于解释“模型为什么给出当前的预测值”这一静态层面。在真实的工业与决策场景中，如果对部分特征进行数值修改，那么如何直观地对比修改前后特征影响力的具体变化，此外SHAP 在存在多重共线性的特征中会出现的微小偏差[21-23]，目前业界还缺少一套标准化的动态对齐分析流程。

1.2.3 大语言模型与多目标进化算法现状

在处理多变量调度以及参数优化的问题时，NSGA-II [24] 和SPEA2 [25] 这一类基于种群进化的多目标算法获得广泛使用[26]。为解决业务人员提出的自然语言需求无法直接转化成底层算法参数的问题，近年来有一部分研究开始尝试让大语言模型（LLM）来解析语句，从而提取出文本中的逻辑约束条件[6, 27]。但是到实际

的测试中就会发现，尽管能通过少样本提示（Few-shot Prompting）[28]的方式规范模型的输出，然而大模型处理那些极其严格的数值物理边界或者是绝对锁定的限制条件时，依然很容易引发逻辑冲突、机器幻觉甚至是JSON 格式的错误[29-30]。另一方面，即使已经成功将这些限制条件转换成底层的代码结构，传统的NSGA-II 算法在进行交叉变异操作的时候，也容易破坏这些由实际业务界定好的严格约束[31]。这种情况会导致算法在每一轮迭代时都产生大量不合法的废弃解，不仅浪费计算资源，甚至连一个可行的方案都无法给出。因此，本文引入带有局部修复能力的拉马克机制（Lamarckian Mechanism）[32-34]，来解决这类带有约束条件的优化问题。

1.3 本文主要工作

为了解决工业控制以及金融风险评估中存在的模型难以让人理解、文字需求很难被转换和在强约束条件下寻找最优解困难等问题，本文实现一套将预测、解释和优化功能都集成的系统。本文的主要工作以及核心贡献，主要概括成以下这四个方面：

（1）构建多变量预测模型并进行性能对比；考虑到表格数据具有特征维度高以及变量关联极其复杂的特点，本文实现建立在自注意力机制之上的Tabular Transformer 模型，与传统方法相比，该模型能更好地捕捉变量之间的相互影响；同时本文还搭建多层感知机（MLP）[35]以及XGBoost 模型作为对照组，并在不同规模的数据集上进行相关测试，这就验证本文所采用的模型在处理复杂特征时的预测稳定性，从而为后面的优化环节提供一个较为可靠的数据输入。

（2）构建特征贡献度对比的解释机制。由于深度学习模型在可解释性方面一直存在一些不足，系统便添加SHAP 解释分析功能，该功能不仅量化优化之前各个特征对预测结果的贡献度，还计算优化之后的各个特征贡献度，并且程序通过对比前后的数据来判断给出的优化方案是否可信。系统还采用Align@K 对齐度指标，该指标专门用来衡量系统建议更改的特征是否与预测过程中起核心作用的那些特征变量保持一致，进而将模型的预测结果与最终决策结合在一起。

（3）实现自然语言指令的解析以及防错拦截。由于非结构化的业务口语很难被规则直接解析成数学方面的限制条件，系统借助大语言模型的提示词工程，将用户的文字需求转换成标准的JSON 约束字典。针对大模型输出结果不够稳定的问题[36]，系统在中间层增加了一段本地规则检查逻辑，针对检测出来的区间冲突或格式异常，那些非法指令在进入到底层求解器之前被有效拦截下来，从而保证从前端交互到后端运算的整体可靠性。

（4）改进约束条件下的多目标寻找最优解算法。在遇到变量被强制锁定或能修改的数量受到限制的时候，常规的NSGA-II 算法往往会产生大量不可行解，为了解决这个问题，底层的算法引入拉马克进化机制，并且设计个体强制修复和数值回写机制。在进行适应度评估之前，这个算子会主动重置那些越界个体的参数，从而让

整个种群在合法的物理限制范围中进行演化。实验结果表明，这种做法显著降低无效迭代带来的算力消耗，让系统在极端业务限制的情况下仍然达到百分之百的求解成功率，这使得它具备非常良好的工程落地条件。

3. 2相关方法与基础理论

AI特征值: 6.9%

AI特征字符数 / 章节(部分)字符数: 350 / 5107

片段指标列表

序号	片段名称	字符数	AI特征		
1	片段1	350	显著		6.9%
2	片段2	369	疑似		7.2%
3	片段3	240	疑似		4.7%

原文内容

2相关方法与基础理论

本章主要介绍该系统涉及到的相关基础理论。首先，介绍本系统中用到的几种预测模型，包括多层感知器、Tabular Transformer以及XGBoost。接下来简单说明注意力机制在建模中的作用。然后介绍SHAP方法以及它的基本原理。最后，对多目标优化算法NSGA-II进行介绍，并结合大语言模型，讨论其在自然语言解析和约束条件下优化的应用方式。系统架构图如图2-1所示。

图2-1 系统架构图

2.1 预测模型相关方法

在多变量的关联预测任务中，预测模型对原始数据进行拟合的具体程度，在很大程度上直接影响后续优化调整建议的效果。考虑到本文所涉及的工业与金融数据往往具有维度特别高、特征之间关系极其复杂的特点，为检测不同预测模型在不同场景下呈现出的预测效果，本文引入三种不同的模型并进行一次对比实验，其主要目的就是通过测试多个性能指标，从而挑选出最合适的预测模型。

2.1.1 多层感知器

多层感知机（Multilayer Perceptron, MLP）是深度学习模型中结构最简单，最基础的一个预测模型，本文将其作为对照组来使用。该模型仅使用全连接的权重矩阵和非线性激活函数，完成从原始特征变量到目标函数的预测模型构建。MLP的优势在于它的结构简单、参数量较少并且运行速度非常快。虽然它在处理特征之间那些隐藏较深的关系时，不如注意力机制那样灵活，但它能提供一个较为稳定的预

测基准模型，从而给出一个明确的对照，用以判断仅依靠全连接层能将准确度做到何种程度[37]。它具体的数学表达如公式2-1，2-2 所示：

(2-1)

(2-2)

其中，x用作输入的向量，对应的符号W和b分别表示相关的权重矩阵和偏置项，f和g分别为激活函数。

2.1.2 Tabular Transformer

传统的深度学习模型在处理表格数据时，往往会忽略各个特征变量之间存在的关联。为了解决这种高维度的特征关联问题，本文采用Tabular Transformer 的架构。该模型建立在自注意力机制（Self-Attention）的基础之上[38]，它模仿人类处理信息时的做法：一方面，原始的变量会通过嵌入层（Embedding）直接转换成高维向量。另一方面，等数据送入多头自注意力（Multi-head Self-Attention）模块之后，该模块就会自动为每一个特征分配不同的“权重”，并提取出特征之间的交互逻辑，从而清晰地展示出哪些特征之间绑定较强的关联。完成这些步骤之后，那些经过注意力加权处理的特征就会到达全连接层那里，进而输出最终的预测值。通过运用这种设计，模型解析高维复杂表格的能力显着提升。

6.9%(350)

2.1.3 XGBoost

XGBoost是一种以梯度提升（Gradient Boosting）思想为基础的树模型，在处理表格数据时往往具有非常优秀的能力[13]。本文挑选该算法作为传统机器学习方向上的一个基准模型。XGBoost最核心的运行逻辑就是拟合残差：它通过训练多棵树的方式，让每一轮新生成的树拟合上一轮剩下的残差。在处理具体表格数据的过程中，XGBoost往往能获得较高的准确度，加上其训练所需的时间又较短，所以经常作为处理表格数据的主力模型。

2.2 注意力机制

在处理表格数据时，模型通常会将各个输入特征当作完全独立的变量来处理。然而在多维度的变量之间往往存在着非常复杂的联系，各个变量会相互影响，这种运算逻辑无法发现变量之间深层的关系。因此本文在预测模型中引入自注意力机制（Self-Attention），该机制在计算过程中会对特征之间的相互关系进行数学建模，从而有效增强对数据的预测能力[38]。

这种注意力机制的核心思路在于，它可以通过计算输入特征之间的相似程度，为不同的特征分配完全不同的“关注权重”。特征之间的关联越强，分配到的权重就越高，到计算的时候就会更加注意它们之间的那些联系。至于它具体的计算过程，则是通过对输入向量做线性变换，从而完整构造出查询（Query）、键（Key）以及值（Value）这三个特征矩阵。模型通过计算Query和Key的乘积来评估特征之间的相关程度，其对应的数学计算公式如公式2-3所示：

(2-3)

虽然常规的注意力机制大多数用于处理像自然语言这样的数据，但本文处理的是一批带有连续型以及离散型变量的工业与金融表格数据。为了让模型能有效处理这一类表格数据，本文在系统整体架构上参考TabTransformer的设计思路[12]。具体操作时，首先通过嵌入层（Embedding）将所有原始特征统一映射到维度完全相同的高维空间中，完成这些之后才会送入多头自注意力模块。

采用这种架构之后，预测模型在进行训练迭代时就会自动找出那些影响较大的特征组合，并将数据内部隐藏的变量交叉关系直接转换成一系列具体的权重矩阵。通过使用这种处理方式，不仅使模型拟合高维数据的能力显著提升，也可用来将神经网络中极其复杂的非线性关系转化成带有具体数值的参数。到后续进行模型建立以及可解释性分析的时候，这些被明确记录下来的权重数值自然就成为一套较为可靠的计算依据。

7.2%(369)

2.3 SHAP解释方法

深度学习模型虽然在多维变量预测中具有高精确度的特点，但是内部处理的结构层次较多，使得模型内部计算过程难以理解，具有“黑盒”的属性[5]。在面向工业调度与风控等真实业务场景，如果缺乏推导逻辑，预测输出很难取得工作人员的信任，从而导致模型的预测结果难以被采纳的问题。为解决算法精准度与信任度的问题，本节使用SHAP（SHapley Additive exPlanations）算法[18]，来计算各特征对结果的贡献度，使得模型预测结果不再是无依据的数值，而是各个特征贡献的叠加结果。

2.3.1 Shapley 值的基本思想

SHAP方法的理论基础直接建立在合作博弈论中的Shapley值之上[17]。当处在多变量预测的实际场景中时，可以将模型最终输出的预测结果视为所有特征一起“合作”之后产生的产物，每一特征的贡献度累加得到预测结果。SHAP方法核心的逻辑，是它会分别计算某一个具体特征在各种不同组合情况下的边际贡献，然后取这些贡献的平均值，从而利用该数值衡量该特征对最终结果产生的实际影响程度。而它核心的计算公式如公式2-4所示：

(2-4)

其中， N 为全体特征集合， S 为不包含特征 j 的特征子集， $v(S)$ 为子集 S 的模型预测值。

2.3.2 加性归因特性

SHAP具备一个称为可加性的特性，具体来说就是所有特征的贡献值的总和，会等于模型预测值与基准值之间的差值。正是基于这种特殊性质，各个特征产生的具体影响就能比较直观地呈现。在系统开发的实际过程中，本文并没有直接计算SHAP的数值，而是针对那些实际的工程需求，做出两项较为关键的改进与设计：

第一项，由于复杂模型计算SHAP值耗时过长，容易拖慢系统响应速度[39]，本

文参考KernelSHAP的加速思想[18]，在预处理阶段引入基于K-Means的数据压缩策略，也就是通过K-Means方法将全体数据划分为几个具有代表性的集合，然后提取背景数据集的聚类中心来替代全量样本参与计算，这样在保证解释精度的前提下，大幅降低计算贡献度所需的时间。

第二项，常规的解释方法通常只会对模型预测出来的结果做一次单独的贡献度计算[21]。而在本系统中，为体现优化建议是否有效改进预测结果，这个模块专门对优化前后的各个特征贡献度进行对比计算。程序在运行期间会提取相应的参数，从而算出调整前后的各个特征贡献度差值。通过对比这组数据，系统就能清楚标示出整个优化过程改动哪些核心变量。展现出这种具体的数值变化记录之后，那些给出的优化建议就拥有较高的可信度，现场工作人员也能理解改变参数会给结果带来什么影响和具体数值变化多少，从而有效建立一线业务人员对模型预测结果的信任感[40]。

2.4 多目标优化与NSGA-II 算法

在预测模型的建立以及特征贡献度解释的工作顺利完成之后，系统最终需要依据当前的业务状态输出一些切实可行的调整建议。不过考虑到许多表格数据往往存在多个不同的输出结果，这就要求必须将这些输出结果很好地平衡起来，因此本文引入多目标优化的方法[41]，其目的在于帮助用户找出最合适的解。

2.4.1 多目标优化问题

在工业调度或者风险评估的实际场景中，进行优化的任务往往带有多个不同的目标，并且这多个目标之间通常又存在一些互相制约的冲突；以建筑能耗的数据集为例，制冷能耗和制暖能耗之间就存在一定矛盾，压低制冷能耗往往就会连带拉高制暖能耗，因此很难找到一组能同时让这两个指标都处于最低状态的参数组合。

对于这类问题，一般不存在一个让所有目标都达到绝对最优的唯一解，而是得到一组互相之间无法完全替代的解集。这些解在不同目标之间形成一种平衡状态，称为帕累托最优解（Pareto Optimal Solutions），它们整体在目标空间中构成的边界则称为帕累托前沿[42]。本系统的任务，就是在这个前沿面上搜索出符合实际业务逻辑的解，让操作人员根据现场情况进行选择。

2.4.2 NSGA-II 方法

为高效找出那些藏在复杂空间中的最优解，本文选用带有精英保留策略的非支配排序遗传算法（NSGA-II）作为计算工具[24]。该算法核心的设计思路主要体现为以下三个方面：

1. 非支配排序，算法会根据每一个个体在各个目标上表现出来的性能进行分层，若有一个方案在所有目标上都不会弱于另一个方案，并且至少在一个目标上表现得比对方更好，则该方案就会被系统视为更优秀的解。运用这种分层逻辑之后，系统就能优先保留那些综合表现较为出色的方案。
2. 拥挤度计算，当面对处于同一个层级中的多个方案时，算法会计算个体与其

4. 7%(240)

他方案之间的具体距离，从而评估它们在空间中的分布情况，计算出的分布越分散，就意味着给出的建议方案越具有多样性，这样便避免最终决策结果过于单一的问题。

3. 精英保留策略，在每一代的进化过程中，程序会先将上一代的父本个体与新产生的子代个体合并到一起，然后再进行一轮筛选。筛选时会先按照已经分好的层级依次挑选，当挑到某一个层级却发现已超过预设的数量限制时，系统会结合前面算好的拥挤度继续挑选，一直到满足所设定的种群规模为止。

2.5 大语言模型与意图解析方法

大语言模型具备较强的非结构化文本处理能力。本系统引入该技术，核心目的就是让机器能够直接读懂用户的自然语言指令，并将其精准转化为优化算法可执行的结构化参数[6]。

2.5.1 大语言模型基础

大语言模型大多建立在Transformer架构上[38]。因为这些模型在预先训练的时候输入大量的数据，所以在应对那些口语化或者模糊的表达时，就会比传统的规则匹配算法表现得更加灵活[27]。考虑到实际业务中工作人员输入的文字指令通常没有什么固定的格式，如果拿常规那种提取关键字的方案来用，一方面会让规则的编写过程变得非常繁琐，另一方面也很容易发生漏解析掉关键信息的情况。因此本系统就专门引入大语言模型进来，主要是想依靠它理解上下文的能力，直接跨越那些不规范的句式障碍，从而从文字中完整提取出具体的变量以及各种限制条件。

2.5.2 提示词与上下文使用

本文在系统中主要依靠提示词工程（Prompt Engineering）[30]来调度模型。具体做法是将任务背景、所需的输出格式以及少量标准示例打包进对话上下文，通过少样本提示（Few-shot Prompting）[2]引导模型进行信息提取。在以往的系统交互中，如果想要限制某些变量，用户必须在软件界面上逐一配置参数范围，操作较复杂。现在借助这套解析流程，系统可以直接从一段业务自然语言描述中，自动拆解出“哪些特征必须锁定”、“数值允许调整的范围”以及“最多允许改动几个参数”等关键信息[43]。这些经过结构化解析的条件，最终会直接作为参数传给后端的优化求解器。

2.6 本章小结

本章详细梳理系统中会涉及到的那些基础方法，包括预测模型、特征的建模方式、解释的具体方法和优化算法等多个方面的内容；一方面是文章的前半部分，主要是围绕着怎么给多变量的数据建立模型以及怎么解释结果来展开的，另一方面则是文章的后半部分，详细介绍在带有限制条件的情况下进行优化求解的相关方法；上面提到过的这些理论内容，给系统后续进行整体架构的设计以及实验数据的分析，提供一个非常扎实的基础支撑。

4. 3多变量预测模型的构建与实现

AI特征值: 15.3%

AI特征字符数 / 章节(部分)字符数: 517 / 3386

片段指标列表

序号	片段名称	字符数	AI特征		
4	片段1	316	疑似		9.3%
5	片段2	517	显著		15.3%

原文内容

3多变量预测模型的构建与实现

在工业和金融等实际的应用场景中，模型刻画清楚多个变量之间复杂关系的能力，在很大程度上决定模型最终预测出来的准确度；因此本章会详细介绍一下系统中预测模型的设计思路以及具体的实现过程[44]，这其中一是包含数据的具体组织方式以及特征的预处理步骤，二是涵盖不同类型模型的构建方法，三是则会完整列出模型训练与评估所用到的那些具体指标。

3.1数据处理流程与特征预处理

在真实业务数据中，各个特征往往存在量纲差异大、分布不一致等问题。为防止因为不同特征数值取值范围不同，导致模型关注具体数值大的特征变量，使得预测精度下降，影响收敛效果。因此，在建立模型之前，需要对原始数据进行必要的预处理[45-46]。

3.1.1 特征与目标的动态划分

在进入模型训练环节之前，程序会完整解析输入的原始数据表，并为所有的列字段生成对应的索引。考虑到不同业务的数据格式往往存在一些差别，系统在底层代码中没有固定输入和输出的维度。面对一张总共包含K个字段的数据表，能够直接通过配置参数的方式，将其动态切分成N个输入变量以及M个预测目标。运用这种比较灵活的切分机制之后，系统仅靠同一套处理逻辑，就能很好地兼容单变量预测以及多目标联合输出的业务场景。

3.1.2 最大最小归一化

为将不同特征因为取值范围不同而对模型训练产生的影响减小到最低，本文采用最大最小归一化的方法，将全部数据统一压缩到[0, 1] 这个区间中[47]。对于任意一个特征向量x来说，它具体的转换方式如公式3-1所示：

(3-1)

在这个公式中，和分别代表这个特征在整个数据集中的最小值以及最大值。在

9. 3%(316)

具体的代码实现时，系统会保留归一化过程里用到的所有参数[47-48]，当输出预测结果的时候，就能使用这些参数重新恢复到最原始的数据范围中。

在挑选数据预处理方法的时候，本文之所以选择这种最大最小归一化而没有用 Z-score 标准化[47,49]，主要是为能够配合到后续第五章中大语言模型输出出来的那些限制条件。因为归一化处理过后的数值和原本的物理数值之间存在着一种很明确的线性映射关系，这就给系统在优化过程里进行反归一化操作提供显着的便利，从而让一线的业务人员能够直接使用真实的数值来表达自己的实际需求。预处理流程如图3-1 所示：

图3-1 数据预处理流程图

3.2 各类预测模型的构建

为比较清晰不同方法在处理多个变量的建模任务中表现出的实际效果，本文同时搭建传统的机器学习模型以及深度学习模型，并将它们作为对照。

3.2.1 XGBoost

作为一种专门用来对比的方法，本文引入XGBoost 模型[7]作为传统方向上的参照。该算法通过迭代拟合残差生成决策树，在处理表格特征时能获得较高的准确度。为保证后续对比实验不会受到无关变量的干扰，测试时对一部分核心运行参数做出硬性规定，一是将迭代次数统一固定为100，二是全局锁定随机种子，以确保每一次运行得到的测试指标都完全一致。选用XGBoost 来作为传统机器学习的参照基准，是因为它自带的正则化功能可以很好抑制过拟合[50]，并且它提取特征交叉信息的能力会远超普通决策树。 XGBoost 的架构图如图3-2 所示：

图3-2 XGBoost架构图

3.2.2经典多层感知机

除树模型之外，实验还基于PyTorch 框架[44]构建了一个标准的全连接神经网络（MLP）[35]，并以此作为深度学习组中的基础对照。它的前向传播过程会相对简单，主要是依靠隐藏层的线性堆叠以及激活函数来转换数据。虽然增加层数能让它具备一定程度的非线性表达能力，但受限于全连接这种固有结构，代码内部并没有专门设计用于提取特征交互的模块。所以当遇到维度较高并且变量关系错综复杂的表格数据时，这种基础网络的拟合能力通常存在比较明显的瓶颈。该对比网络的基础结构有以下的设置：

- 1. 输入层：接收归一化后的N维特征。
- 2. 隐藏层：由多层全连接结构构成，使用ReLU激活函数[51]。
- 3. 输出层：将特征映射到M维目标空间。

多层感知机的架构图如图3-3所示：

图3-3 MLP架构图

3.2.3 Tabular Transformer

相比传统的全连接网络，基于注意力机制的模型在处理特征之间的各种关系时

会更有优势。为此本文实现一个面向表格数据的Tabular Transformer 模型[12]，用来计算各个特征之间的关联。该模型首先会通过一个特征嵌入层，将原始特征转换成高维向量，这样它们就能参与到后续的注意力计算中[52]。之后还会引入多头自注意力机制[38]，对不同特征之间的关系进行建模，同时结合残差连接与层归一化，以提高训练过程中的稳定性[53]。

(3-2)

经过注意力层后，特征进一步输入前馈网络进行非线性变换：

(3-3)

最终，通过特征展开和全连接层输出预测结果。

Tabular Transformer的架构图如图3-4所示：

图3-4 Tabular Transformer架构图

3.3模型训练与参数设置

在模型训练过程中，因为数据集往往比较大，如果一次性使用全部数据做更新，就会遇到内存不足的问题，进而导致训练的不稳定。所以本文在实现的时候用基于PyTorch [44]的Mini-Batch 训练方式，这样能同时兼顾到训练效率和稳定性[54]。

3.3.1 数据加载与批处理方式

在数据预处理阶段，系统会先将那些归一化处理过的特征变量和标签封装到一起，然后再按照批次读取数据。因为综合考虑不同规模的数据会对显存产生不一样的影响，所以每一批的数据大小就设置如公式3-4所示：

$$\text{Batch Size} = \min \setminus (64, \text{Total Samples} \setminus) \quad (3-4)$$

在每一轮的训练过程中，这些数据都会被程序随机打乱。运用这种打乱机制之后，就能避免模型过度依赖样本的先后顺序，从而顺利提高模型的泛化能力。

3.3.2 优化器与超参数控制

至于在挑选损失函数的时候，本文采用均方误差，并且将其作为最终的训练目标。因为这个指标在面对误差比较大的样本时会表现得非常敏感，所以被认为非常适合回归任务。不过不同的模型在真正进行训练的时候，所采用的参数配置还是会在一些差别的：

1. 当面对结构相对来说比较简单的MLP模型时，程序会使用一个比较大的学习率（ $\eta = 0.005$ ），并且将训练的轮数设定为100轮，其目的就是为加快算法收敛的速度。

2. 针对Tabular Transformer预测模型，由于其架构复杂，参数多于基础模型，系统使用Adam 算法更新网络梯度[55]。为了避免模型在参数寻优的过程中，出现剧烈震荡，初始的学习率设定为 $\eta = 0.001$ 。与此同时，迭代次数也增加到500轮，这样做主要是希望通过增加周期，让网络在后期更平缓也更稳定地完成收敛。

3.4预测性能评估指标

15.3%(517)

在整个模型完整训练完成之后，系统就会在验证集上进行测试，并且会使用下面几个具体的指标来对最终预测出的实际效果进行详细评估[56-57]：

1. 均方误差（MSE）：即先算出预测值与真实数据之间的差值，再求出这个差值的平方以及整体的平均值，其计算公式如公式3-5所示。

(3-5)

2. 平均绝对误差：即先计算出预测值与真实数值相差的绝对值，接着再求其平均值。与前面的MSE相比，MAE对异常数据的敏感程度更低一些，因此能更真实地反映整体的误差水平，计算公式如公式3-6所示。

(3-6)

3. 决定系数：它用于描述模型对数据变化的拟合能力的强弱，其取值范围通常限制在0 到1 之间[58]。系统最终算出的数值越大，就表明整个数据拟合的效果越好。计算公式如公式3-7所示。

(3-7)

3.5本章小结

本章详细梳理多变量预测模型的完整实现步骤，其中包含数据预处理、模型参数的设定以及训练与评估等内容。一是模型选择方面，系统同时引入XGBoost、MLP和Tabular Transformer这三类方法，并且在统一的数据处理流程之下做对比。二是还对训练过程中所采用的批处理方式以及几个关键的参数设置做说明。上述这些内容能为后续的解释分析与优化模块提供一定的基础。

5.4基于 SHAP 的模型解释方法

AI特征值: 0.0%

AI特征字符数 / 章节(部分)字符数: 0 / 4028

片段指标列表

序号	片段名称	字符数	AI特征
----	------	-----	------

原文内容

4基于 SHAP 的模型解释方法

前文构建的Tabular Transformer[12] 虽然在测试集上达到预期的拟合准确度，但受限于深层网络的复杂的运算流程，系统输出的预测数值缺乏直观的逻辑支撑。在真实的工业调度或金融风控现场，如果仅仅给出一个黑盒预测结果而无法提供具体的判断依据，业务人员通常不敢轻易采纳。为打破这种不透明的状态，本文在预测流程之后专门接入SHAP 特征解释模块[18]。该模块不参与预测，而是负责解释模型得出该结论的具体原因，各个特征对结果具体贡献量。考虑到实际业务的需求

，系统没有单纯停留在解释单次结果上，而是增加优化调整前后的特征对比功能。本章将详细梳理这一分析模块的具体实现过程，重点记录特征贡献分数的计算提取，以及如何利用这些分数来论证最终决策方案的合理性。

4.1 SHAP计算方法与实现

因为本文采用的模型结构设计得比较复杂，如果直接使用最完整的训练数据计算SHAP值，往往就会引发较大的运算负担；所以在系统真正实现的环节中，相关的计算流程专门进行了简化以及优化的处理，通过运用这种调整，就能兼顾计算效率以及最终结果的稳定性。

4.1.1 基于 K-Means 的参考数据简化

在SHAP 算法计算边际贡献的时候，需要先选出一组参考数据来充当基准。如果直接拿全部样本用，就会带来极其漫长的计算耗时，所以本文采用SHAP 框架提出者Lundberg 所给出的降维思路[18]：一旦参考数据集变得过大，就能用少量具有代表性的数据近似代替全局分布，这样能有效减少计算量[39]。基于这一点，本文对标准化之后的特征数据做K-Means 聚类处理[59]，并且从中选出少量有代表性的聚类中心来充当参考数据，计算公式如公式4-1 所示。

(4-1)

通过采用这种处理方式，系统能在一定程度上大大减少运算时的计算量，并且同时还会尽量保留原始数据所带有的那些分布特征，从而非常顺利地平衡运算效率以及预测精度之间，达到较好的平衡。

4.1.2 多输出模型的处理方式

因为本文采用的预测模型最终输出的结果是多维的向量，然而SHAP 方法在通常的情况下往往只会针对单一的输出变量进行分析。所以为解决这个矛盾，在代码具体实现的过程中，程序专门对模型输出的结果进行一次拆分处理[60]，这其实就是将每一个拆解开的输出维度单独去除，然后再分别对它们进行详细的解释。而它最后呈现出来的具体形式如公式4-2 所示：

(4-2)

采用这种处理方式之后，系统就能分别分析出不同目标变量的具体影响因素，这就很好地避免多个输出之间容易产生相互干扰的情况，从而让最终解释出来的结果变得更加清晰明了。SHAP的计算流程如图4-1所示。

图4-1 SHAP流程图

4.2 当前状态的特征影响分析

在进行参数的优化之前，程序需要先对这组初始输入的特征进行一次贡献度计算。利用SHAP方法，可以算出每一个变量当前分配到的贡献权重，然后系统就能借助这些算好的数值，将当前预测结果的具体原因给出清楚的解释。为让最终结果更易于使用者理解，程序还会将算出的SHAP值按照预测差值做一定比例的映射处理。假设将当前的预测值设定为某值，并且将基准值也设定为某值，则两者之间的差值

如公式4-3所示：

(4-3)

对应的特征贡献调整如公式4-4所示：

(4-4)

根据计算出来的贡献值，能够对所有的特征进行排序，并且还能将那些影响比较大的变量专门挑选出来进行重点分析。同时，如果结合瀑布图[61]，就能更加直观地将各个特征对结果产生什么方向的影响以及影响的具体程度给完整地展示出来。

4.3 决策结果与关键特征的关系分析

在对模型预测结果的解释工作完成之后，系统就会自动进入到多目标优化的环节中。为评估优化算法给出的参数调整方案是否具备实际的指导意义，即判断算法在调整时有没有准确识别那些对预测结果影响最大的特征变量，本文专门引入决策一致性指标[62]，系统会直接使用这个指标来计算那些建议修改的特征与那些起到核心贡献的特征之间，有多高的重合程度。

首先，确定优化前后发生变化的特征集合：

其次，系统就会根据SHAP所计算出来的那些分析结果，选出那些贡献值比较大的前K个特征：

其中K的值如公式4-5所示：

(4-5)

在这里，之所以将K的取值设定成一个不小于三的数字，主要是为了保证系统运行时的稳定性，并且能让它更加贴合实际情况。在系统最开始进行架构设计的时候，直接将K的取值设置成为那些修改的变量数量。然而在工业能耗数据集中进行测试的时候，这种绝对匹配的逻辑却直接暴露一些问题。

通过对后台输出的日志进行详细检查之后就会发现，在真实的物理环境当中，有一部分核心变量之间存在着极高关联性的[63]。正是这种强烈的关联性，导致它们计算出来的SHAP贡献分数往往会变得很接近，排在前三名的权重差值有时仅仅相差在小数点后面的第三位。在这种特殊情况下，如果指令严格（例如‘仅允许调整1个参数’）且影响最大的特征被业务规则锁定，优化引擎将放弃调整该特征，转而修改排名第二或第三的关键变量。

对齐度计算流程如图4-2所示：

图4-2 对齐度计算流程图

从工程角度看，这种“次优替代”属于合理且必要的让步调整。但在系统初始的严格评价逻辑下，由于没有准确调整特征贡献第一名的变量，系统的对齐度会被判定为0%。为消除这种因约束条件和变量相关性带来的错误判断，本文在代码逻辑中为K的计算设定一个容错窗口。

在公式（4-5）中之所以将最低阈值设定为常数3，是综合考虑到实际业务的调

度习惯以及算法本身的适应性的结果。因为程序在校验层面上增设一套容错机制，只要优化算法最终挑选出来的变量属于全局权重最高的前三名核心变量，系统就认为当前的优化方向较为合理。这种允许存在一定偏差的设计，一方面有效应对相近SHAP权重所导致的问题，另一方面也能让系统在较为严格的限制条件之下，给出更加客观的性能评价。

最终计算两者的重合比例计算如公式4-6所示：

(4-6)

这个计算出的指标就能很好地反映出优化过程中所调整的变量与那些影响较大的特征变量之间的对应程度。

4.4 优化前后的结果对比分析

那些常见的解释模块通常只会计算当前状态下的单次特征贡献度，但是在真实的业务调度中，这种单纯的结果无法体现出参数修改之后产生的后续影响，而且工业现场又要求算法的决策过程必须做得非常清晰透明。针对这个比较严格的要求，程序保留原先那个初始的分析功能，并且在后面追加一道对特征贡献度进行动态比对的步骤。后台系统会分别完整提取出算法调整前后的那些变量权重，然后再算出它们之间的差值。利用这组差值数据，程序就直接量化优化动作对关键因素的实际调控程度[64]。这种前后数值比对的做法，很顺利地解决单次解释容易存在盲区的问题。等获得这批对比过的数据之后，系统最终输出出来的调度方案就具备更加可靠的客观依据。

4.4.1 特征贡献变化分析

等到寻优引擎输出推荐的方案之后，系统就会自动触发第二次的SHAP解释逻辑。因为模型内部的参数保持一致，所以这两次测得的贡献值就具备较强的可比性。为把关注点聚焦在决策行为本身上，分析模块就会自动过滤掉那些没有发生过变动的环境变量，仅仅只针对那些经过算法修改的特征集合进行对比。分别计算出优化前后的SHAP值，并且仅保留发生过变化的特征，然后再算出它的差值，如公式4-7所示：

(4-7)

因为实时计算特征贡献度所消耗的时间会比较长，所以在代码层面就引入基于Key-Value结构的内存缓存机制。当优化算法输出新的参数组合时，系统并不会一直等到计算结果完成才计算SHAP的值，而是让后台程序异步更新SHAP的权重。运用这种类似于“用空间换取时间”的缓存逻辑之后，程序在给出最终的参数调整建议时，就能直接从内存中取出特征对比的数据进行展示，这保障实际业务在交互时候的实时性，从而避免让用户长时间的等待[39,65]。贡献度对比图如图4-3所示：

图4-3 贡献度对比图

4.4.2 结果对比与分析

观察数据的分布情况之后，决策者就能理解优化算法具体的调整方向。比如在

优化建筑能耗的场景中，若初始诊断将某个特征判为主要提升能效的反向因素，那一个合理的优化方案就应当显著降低该特征在优化之后的反向影响程度。为此系统构建一套完整流程，主要包含三个环节：一是事前诊断，即计算各个特征对预测结果的贡献度，并以此为依据挑选出那些对输出目标影响最大的特征变量。二是检测对齐，即检测优化算法最终决定要调整的特征变量，判断所做的改动是否与前面挑选出的重要特征变量相吻合。三是前后对比，即提取调整参数前后的各个特征贡献值进行比对，主要用于验证那些起主要作用的特征变量在优化之后是否得到有效改进。运用这套解释流程之后，深度学习模型内部那些复杂的映射关系就直接转换成清晰的参数修改记录。从实际测试结果来看，若系统能提供底层数据并列出明确的调参理由以及调参之后的数值变化，一线人员在使用该系统时的顾虑就会大幅减少，这种做法解决仅凭预测结果无法直接指导实际调度的问题[66]。此外，基于这批对比过的数据，在后续第五章验证算法在约束条件下搜索结果的合理性的时候，也就有一个明确的参照基准。

4.5 本章小结

本章详细梳理SHAP特征归因模块在决策系统中的完整实现过程。针对背景数据过多降低运算速度的问题，代码层面采用基于K-Means的聚类降维策略，从而有效保证解释引擎在实际业务中的运行效率。完成基础变量归因之后，本章并未局限于单次静态的结果解释，而是专门构建参数调整前后的特征影响比对机制。运用这套检验逻辑之后，原本不可见的黑盒预测过程就转化成非常直观的数值参考，这也为下一章接入带有复杂限制条件的多目标参数寻优算法，提前准备可靠的逻辑验证工具。

6. 5结合大模型的受限决策优化方法

AI特征值：0.0%

AI特征字符数 / 章节(部分)字符数：0 / 3696

片段指标列表

序号	片段名称	字符数	AI特征
----	------	-----	------

原文内容

5结合大模型的受限决策优化方法

实际业务场景中的需求通常由用户以自然语言的形式提出来，例如“设备A的状态保持不变，最多微调3个参数来把能耗降到最低”。传统的规则引擎没有办法直接读取这一类非结构化的文字条件。除此以外，当业务需求中包含变量被绝对锁定、或者修改数量被设定上限（L0稀疏约束）这一类强限制的时候，基础的启发式算法

在做完交叉和变异操作之后，很容易生成大量越界的无效解。这就会导致种群在进行迭代的时候陷入到死锁状态里，从而无法计算出任何能符合现实要求的方案。为彻底解决这些问题，本系统在前端利用大语言模型将文字指令解析成结构化的数学边界，在后端则深入到NSGA-II 的底层逻辑中，实现针对强限制条件的子代强制修复算子。这套组合起来的机制很有效地解决常规优化算法在面对极端业务限制时容易失效的问题。

5.1 基于大语言模型的意图解析与约束转换

为实现自然语言到数学表达的转换，系统接入大语言模型（LLM）作为中间的处理模块，其任务就是将用户输入的日常工作文字转化成程序能直接读懂的条件参数。在具体的实现过程中，主要是通过构建出一套相对固定的系统提示词，再结合上大模型理解上下文的能力，就将用户输入的文字内容解析成一套统一格式的JSON 结构[2,30]。这个结构主要划分成下面这三类信息：第一类是目标集合，它用于描述优化目标的方向，比如最小化、最大化或者是接近某一个目标值。第二类是状态约束集合，它用于表示目标变量在取值范围上的限制，一般的形式就是区间约束。第三类是特征约束集合，它用于描述变量能够调整的范围，这其中就包括保持不变的变量、允许调整的数值区间，以及最多能够调整的变量数量。经过上面这些步骤之后，原本比较模糊的文字需求就转换成非常明确的限制条件。特别是在本文涉及的金融风控等垂直领域中，大模型展现出的专业文本解析能力为这种转换提供了可靠保障[67]。这样一来，底层的优化算法也就获得一个非常清晰的搜索范围[28,43]。

5.2 约束校验与修复机制

考虑到大语言模型在提取信息的时候偶尔会生成一些不稳定的输出，并且用户提交的原始指令本身也有可能会存在逻辑上的矛盾，比如设定好的下限数值比上限还要大，或者是限制的修改次数变成负数。如果直接将带有这种错误的数据传给后端的优化模块，就极易导致算法陷入到死锁或者程序异常报错。为保障整个调用流程的稳定性，系统在正式启动寻优计算之前，专门加入一条参数前置的校验逻辑。这段代码会逐一检查大语言模型输出的约束字典，将检测出来的异常边界值强制拦截并进行修改。具体的修复规则主要涵盖以下这两种情况：第一种情况是区间修正，当检测到区间约束中存在下限大于上限的时候，程序就会交换它的上下界，使其恢复成正确的区间。第二种情况是稀疏参数的处理，当解析出来的最大修改数量小于0的时候，就直接将其截断为0，从而避免后续计算中出现异常情况。大模型解析流程如图5-1所示：

图5-1 大模型解析流程图

5.3 多目标优化方法设计

在寻找最优解的阶段，系统就采用带着精英保留策略的NSGA-II 算法来当作主要的计算引擎。因为在本文涉及到的建筑能耗调控或者信贷风险评估这些业务场景中，决策过程往往需要在多个互相矛盾的目标中间找出帕累托最优解。不过由于大

模型解析出来的业务限制条件通常设定得非常严格，这就导致标准的NSGA-II 算法应用于实际场景中的时候，会面临非常严重的搜索效率问题[68]，所以就必须对它进行一些专门的改进。帕累托前沿图如图5-2所示：

图5-2 帕累托前沿

系统早期的代码在处理业务端提供的物理边界以及变量锁定规则时，程序直接采用硬约束拦截逻辑，即剔除所有越界个体。但是后台的运行日志显示出这种做法极易触发“初始化死锁”问题，因为实际业务下发的限制条件往往设定得非常苛刻，这直接导致合法的解空间变得极其狭窄。当系统随机生成第一代种群的时候，违规个体的比例经常会超过90%，若在这个环节继续运行强制清理代码，种群的有效规模就会严重消耗，算法库也会因此彻底丢失基因多样性，后续的演化计算只能终止。

为妥善解决这个问题，本文放弃传统的那种“硬过滤”策略，转而引入基于惩罚分数的软约束处理机制。该设计的核心思路是不再丢弃那些越界个体，而是将它们偏离规定的程度转化成相应的惩罚分数，通过“污染”适应度函数的方式，在数学计算层面上赋予这种解一个较低的竞争力[69]。

本文构造的修正后目标函数如公式5-1所示：

(5-1)

在这个表达式中， λ 并不会设置为一个固定的数值。为让算法在开始运行的前期具备比较广阔的探索能力，并且到后期能顺利实现非常精准的收敛效果，底层的代码在具体设置它的时候，就专门采用一种分阶段的方式：

第一个阶段是在进化的初期，程序会取一个比较小的 λ 数值，此时允许部分轻微超出限制边界的个体进入算法参与演化，这么做目的是借助这些越界个体的潜在信息，尝试探索边界外面可能藏有更好结果的区域，从而扩大初期的搜索范围。

第二个阶段是在进化的后期，随着迭代次数不断增加，程序会将 λ 的取值逐渐提高，对于那些明显偏离规定的个体，系统会显著压低它们的适应度评分，在后续执行非支配排序以及选择操作的时候，这些不合格的个体就会被快速淘汰掉。

通过运用这种动态调节权重的策略，算法的搜索重点就会从早期的全局探索转变到后期的严格合规上，这一过程也强制引导着种群向着合法的解空间靠拢[70]。从实际运行效果来看，这种处理流程顺利解决常规求解器在严苛业务条件下容易发生的死锁问题。除此之外，这种“先松后紧”的筛选逻辑也很好地保护底层数据的多样性，从而为第5.4节中提到的“拉马克强制修复”保留足够的候选基因，最终也就保证推导出来的帕累托前沿方案能够严格地与业务红线对齐。

5.4拉马克式修复与回写方法

前文的惩罚项虽然能处理数值区间的状态约束，但遇到前端解析出的“最多只能改N个参数”或者“某些特定特征绝对保持不变”这种强业务限制时，标准的NSGA-II 算法就会表现出明显的不适用性。核心痛点在于，底层自带的交叉和变异

算子通常会对染色体上的所有基因位做随机扰动，这就导致变异产生的子代个体几乎必然会打破“仅微调少数特征”这条规则。种群中一旦充斥大量这类不可行解，算法的算力就会严重浪费。为避开这个陷阱，本文在传统的进化框架中专门增加一段基于拉马克思想的强制修复与回写机制，在个体进入评估前先做一轮数据纠偏。

5.4.1 拉马克思想在算法中的应用

按照常规遗传算法的逻辑，即使在中途对个体进行局部纠正，这种改变往往也只用于计算当前的适应度，并不会真正修改它原本的基因编码（即学术界常说的鲍德温效应[71]）。但在面对本文这种高维数据和严苛业务限制时，只计算分数不更新基因会导致下一代依然变异出大量不可行解。因此，本文的系统直接参考带有“修复并回写”特征的模因算法思路[72]：程序一旦检测到某个个体修改的特征数量超标，或者擅自改动锁定变量，会立刻调用前置规则把它修复成合法状态。最关键的是，代码会将这组修好的合法基因，直接替换掉原来的错误编码，让它带着合法的基因组参与后续的繁衍迭代。

5.4.2 个体修复与回写过程

在NSGA-II 每一轮对个体进行适应度评估之前，系统会对当前个体张量执行如下处理[73]：

1. 计算特征变化幅度：将当前个体与基准状态做一次比较，从而得到各个特征的变化量 ΔX 。

2. 控制变量改动数量：根据 ΔX 的大小进行排序，只把变化幅度最大的K个特征给保留下来，而其余的那些特征则要恢复成基准值，这样就能满足稀疏约束的要求。

3. 处理锁定变量：对于那些用户指定为不可调整的特征，会直接把它们数值重置到基准状态。

4. 个体更新与回写：程序会将经过前置修复的合法参数重新代入模型，算出真实的适应度分数，并用这组干净的数据直接覆盖掉种群里原本越界的错误个体。依靠这一步强制的数值纠偏，整个算法在后续繁衍时，实际上都是在合法解的范围内进行迭代。

采用这种插入式的修复逻辑，最大的好处是完全不需要修改NSGA-II 底层自带的非支配排序和选择算子。它仅仅是在个体算分之前加一层过滤机制，就能在不破坏原算法整体框架的前提下，从源头上阻止无效子代的生成[70]。拉马克回写机制流程图如图5-3所示：

图5-3 拉马克回写机制流程图

5.5 本章小结

本章详细记录系统针对多约束调度环境所做的算法改造过程。从最前端的数据流入开始，一方面利用大语言模型的接口，从自然语言中完整提取出那些严谨的数

学边界，并且配套编写一段前置防错代码，用于提前拦截逻辑上的矛盾。另一方面到后端的优化求解环节，为解决常规算法在遇到很严格的限制条件时容易死锁的问题，系统采用基于惩罚分数的软约束机制，同时还加上针对底层变量的基因强制回写模块。经过这一系列从文本解析到子代纠偏的工程搭建之后，目前系统已经能够在复杂的业务限制之下稳定输出符合规定的方案，而这套改造过的寻优引擎，也会在下一章的对比实验中接受实际数据的测试。

7. 6实验设计与结果分析

AI特征值: 0.0%

AI特征字符数 / 章节(部分)字符数: 0 / 9328

片段指标列表

序号	片段名称	字符数	AI特征
6	片段1	260	疑似 2.8%

原文内容

6实验设计与结果分析

本章通过一系列对比实验，对本文所构建的多变量关联预测与可解释性分析系统进行评估。实验主要从以下几个方面展开：首先，对比核心预测模型（Tabular Transformer）与传统基准模型在不同数据场景下的预测表现。其次，对不同规模的大语言模型（LLM）在自然语言指令解析任务中的效果进行比较。第三，分析SHAP方法在事前诊断与事后对比中的作用，并结合Align@K指标对决策一致性进行量化。最后，在约束条件较强的情况下，对比引入个体修复策略前后，NSGA-II在可行解搜索方面的变化情况。

6.1 实验环境与数据集设置

系统在前端架构中接入一个大语言模型，用于将自然语言指令转成程序可读的结构化参数。该模型的主要作用是从用户输入的文本中提取信息，并按统一格式输出具体的边界约束条件。

6.1.1 软硬件实验环境

因为本课题所涉及的深度网络迭代以及后期的SHAP归因解析会消耗比较大的算力，所以本次实验将所有的代码都部署到带有显卡加速的设备上运行。具体的软硬件基础配置清单有下面这些内容：

计算设备：支持CUDA 的NVIDIA GPU

深度学习框架：PyTorch 2. x [44]

多目标优化工具：Pymoo ，

可解释性工具：SHAP（结合DeepExplainer 与KMeans 降维[18,59]）

大语言模型：基于API 的调用方式

6.1.2 实验数据集

到实验的测试阶段，为验证清楚本文模型在不同业务场景下的适应能力，挑选两个很有代表性的表格数据集。第一个是工业能耗数据集（双目标），这个数据集的结构相对来说比较简单，一共包含8个输入特征，其主要任务是同时预测制热和制冷的能耗，由于它的整体维度比较低，所以各个特征之间存在的关联就显得比较直观。第二个是金融信贷数据集（单目标），该数据专门用于判断客户是否存在违约风险，和前面的能耗数据比起来，它的特征维度出现明显的增加，一共提取到20个像信用记录以及账户状态这样的输入字段，由于特征的数量变得更多，并且数据分布的规律也变得更加复杂，所以就用这个数据集来评估模型在面对复杂表格条件时提取特征的能力。等进入到预处理阶段后，因为原始数据中不同字段的单位以及数值的范围有着特别大的差别，如果直接用原始的测量标准进行训练，模型梯度的正常更新就会受到干扰，因此在读入数据之后就统一执行一次最大最小归一化的操作，将所有特征值都完整压缩到[0, 1]这个区间内。

6.2 不同模型的预测性能比较

为比较不同模型在表格数据上的预测表现，实验选用XGBoost与多层感知机（即MLP）来充当对照模型，并且分别在能耗数据和信贷数据上进行测试。

6.2.1 能耗数据实验结果

在工业能耗数据上，三类模型的训练与测试结果如表6-1所示：

表6-1 能耗场景预测模型性能对比

模型名称	决定系数(R2)	均方误差(MSE)	绝对误差(MAE)
Tabular Transformer	0.9982	0.1652	0.3147
XGBoost	0.9988	0.1142	0.2074
MLP网络	0.9974	0.2438	0.3513

根据测试结果可以发现，参与对比的几个模型在能耗数据集上的均达到0.99 以上，说明整体的预测精度都较高。但如果只看MSE 和MAE 这两个绝对误差指标，XGBoost 的得分是最低的，实际表现略微优于本文构建的Tabular Transformer。

会出现这种结果，主要是受能耗数据本身特征数量较少这一特点的限制。因为这个数据集仅包含少数几个特征，变量之间的逻辑比较直接，并未隐藏复杂的关系，所以当面对这种比较简单的表格时，像XGBoost 这种底层建立在树结构上的模型，仅靠节点分支切分特征的边界，就能高效地划分出局部数据的规律。

反观Tabular Transformer 模型，其内部专门设计出来用于挖掘特征之间深层交互关系的注意力模块，在这种变量关系比较简单的场景下作用有限，反而因为网络层数过深而带来一些多余的计算量。不过值得注意的是，即便自身优势没能完全发挥出来，该模型依然得到与XGBoost 相近的各项误差指标，这就说明即使是在缺

乏复杂交叉关系的简单数据集上，Tabular Transformer 的底层拟合机制也做得足够好，预测精度保持稳定，没有出现明显的下滑。

6.2.2 信贷数据实验结果

为进一步把模型在更复杂特征场景下的表现考察清楚，同样的对比实验在信贷数据上再次进行，具体的测试结果如表6-2 所示：

表6-2 信贷场景预测模型性能对比			
模型名称	决定系数(R2)	均方误差(MSE)	绝对误差(MAE)
Tabular Transformer	0.9879	0.0025	0.0378
XGBoost	0.9182	0.0172	0.0907
MLP网络	0.9505	0.0104	0.0490

在信贷数据集的测试组中，这三种算法之间的性能差距开始出现显著扩大。从具体的测试记录看，Tabular Transformer 的各项误差得分与拟合指标都位列第一，其算出来的综合预测精度显著优于XGBoost 和基础的MLP。相比之下XGBoost 的表现就出现一些下降，因为它的底层代码是沿树结构逐层切分特征，当面对着有效信息高度分散并且高维稀疏的特征空间时，树模型就只能通过被动增加节点的分裂次数来强行靠近真实的业务数据，这种情况就不可避免地降低最终的预测效果。

作为基础对照组的MLP 模型，虽然依靠着多层全连接保留基本的映射能力，但是因为网络结构里缺乏专门处理变量交互的模块，导致最终的测试成绩为中等水平。将这三组数据进行对比就能证实，当业务数据的维度变高，并且字段之间的逻辑关系变得非常复杂的时候，具备特征交叉功能的网络模型，会展现出传统的结构有着更高的预测上限。

6.3 大语言模型的约束解析效果分析

在设计整个前端模块的时候，接入大模型的核心目的就是执行一次文字翻译工作，也就是需要大模型将用户输入的日常文字转换成底层优化算法能够直接读取的规则字典。从具体的代码逻辑来看，这就要求大模型必须同时完成“识别业务意图”与“对齐数学边界”这两个动作。为评估当前主流大模型在这段转换代码里的实际可用性，实验直接复用前文构建好的能耗预测与信贷风控这两个业务场景，并且对五款模型接口进行详细对比。这其中一方面包含有DeepSeek-Chat、Qwen-Max 以及GLM-4 这三款主打推理能力的大参数模型，另一方面也包含有Qwen-Turbo 和GLM-4-Flash 这两款追求响应速度的轻量级版本。这次测试不涉及大模型常规对话功能，仅针对它们输出结构化参数时的准确程度来进行打分。

6.3.1 测试数据构建与评估方法

如果只依赖少量的人工样例，就很难覆盖真实业务中那些复杂又多变的指令表达方式。所以这里没有采用传统那种“手写测试集”的做法，而是采用能自动生成并且自动评估的流程。

6.3.1.1 指令数据的生成与标注

本次评估所用的测试集一共包含200 条指令，整个数据集的构建基本上可以分成两个阶段来完成。

第一，借助大模型来批量生成指令。我们直接提取已有数据集的元信息（如变量名、取值范围这类约束条件），把它们作为上下文输入给大模型，让它模拟真实的业务语境生成自然语言需求。生成的文本，基本覆盖像多目标优化、变量锁定、区间限制和稀疏约束等几类常见场景。指令初稿出来之后，第二步则全部交由人工进行逐条核对，并为每一句话手工标注对应的标准JSON 结构，以此作为后续评测的参考答案

6.3.1.2 评估指标设计

在测试环节，为避开人工核查可能产生的主观误差，模型输出结果的对齐比对工作会全部交给自动化脚本完成。同时为彻底量化出大语言模型的真实解析能力，代码里拆分设计出多个维度的打分规则，一是全局命中率这种最严格的指标，它要求大模型返回的约束字典必须与标准答案完全匹配，哪怕有一个内部条件不匹配也会判定为失败。二是针对局部字段的检验上，脚本设置目标准确率，专门用来检测模型是否把求最大或者求最小的优化方向弄反，同时还利用范围准确率逐一核对底层变量的数值上下界有没有正确提取出来。

另外在处理“提取锁定特征”这种具体的变量挑选任务时，如果只看整体上的对错，就很容易掩盖掉细微的逻辑错误，所以脚本在这一步额外引入精准率、召回率以及这两者的综合得分（也就是F1-Score），以统计特征提取的遗漏与错误。整套组合打分逻辑运行之后，就能非常明确地定位出问题，也就是大模型在解析复杂的业务文字时，到底是JSON的打包格式出问题，还是从根本上就没有理解用户的真实意图。

6.3.2 能耗场景下的解析表现

在工业能耗任务中，指令通常包含明确的物理变量，并涉及双目标优化。测试结果如表6-3所示：

表6-3 能耗场景大模型性能对比

模型名称全局命中率(%) 目标准确率(%) 范围准确率(%) 精准率(%) 召回率(%) 综合得分(F1)

DeepSeek-Chat	86.2	100.0	100.0	86.1	100.0	92.5
Qwen-Max	86.2	100.0	100.0	86.1	100.0	92.5
GLM-4	86.2	100.0	100.0	86.1	100.0	92.5
Qwen-Turbo	83.9	98.8	98.8	85.3	100.0	92.1
GLM-4-Flash	73.6	92.0	97.7	82.7	98.8	90.0

从评测结果来看，DeepSeek-Chat、Qwen-Max 与GLM-4 这几组大参数模型在主要指标上的表现较为接近。在目标识别、数值范围提取以及特征召回率等指标上，三者均达到100%，说明当模型参数规模达到一定水平后，已经能够较完整地提取

业务约束中的关键信息。

不过，从整体命中率来看，这几组模型最终都停留在86.2%左右。进一步分析测试日志后发现，影响该指标的原因并不在于核心语义理解错误，而更多出现在结构化结果生成阶段。部分样例中，模型虽然正确识别约束内容，但在JSON字段的组织方式、键值层级或参数拼接上，与标准答案存在一定差异，因此未能通过严格匹配。

相比之下，作为对照的轻量级模型会在各项指标上出现比较明显的下降。其中的GLM-4-Flash在处理那些约束条件较多且逻辑结构比较复杂的任务时，更容易发生字段遗漏或者变量归属错误这类问题。

综合整体测试结果可以看出，在规则相对明确、变量规模有限的业务场景下，目前主流的大参数模型在解析能力上的差距并不明显，其输出结果已经能够满足后续优化算法对约束输入的基本需求。

6.3.3 信贷场景下的表现变化

当任务切换到金融信贷场景之后，整个情况就有比较明显的变化。这一场景的特点一是变量的数量变得更多，二是数据的分布也更加复杂，同时还存在着大量语义相近的特征名称，这些都会对模型的区分能力提出更高的要求。各个模型的具体表现如下面的表6-4所示：

表6-4 信贷场景大模型性能对比

模型名称全局命中率(%) 目标准确率(%) 范围准确率(%) 精准率(%) 召回率(%) 综合得分(F1)

DeepSeek-Chat	80.0	96.8	97.4	82.5	97.3	89.3
Qwen-Max	80.7	96.1	100.0	82.9	100.0	90.6
GLM-4	83.9	98.7	100.0	82.9	100.0	90.6
Qwen-Turbo	76.1	94.2	98.1	82.4	100.0	90.4
GLM-4-Flash	75.5	97.4	95.5	80.8	95.3	87.5

从具体测试结果来看，几款参评模型的整体命中率相比简单场景都有一定下降。对解析失败的样例进行分析后发现，问题主要集中在特征变量的识别阶段，常见情况主要有两类：一类是模型会出现“幻觉”现象，生成原始文本中并未提及的变量。另一类则是容易混淆名称相近的业务字段，从而影响最终的解析精度。虽然存在上述问题，但参数规模较大的模型整体仍保持较好的解析能力。综合比较后可以发现，GLM-4 在处理长文本以及复杂约束描述时，逻辑拆解能力相对更加稳定。

6.3.4 模型选型与系统补偿策略

结合前文的实验结果，系统最终选用GLM-4来作为默认的文本解析模型，原因主要有两点：一是在普通场景下它的解析准确率与其他模型差异不大，二是在高维特征和复杂约束条件中GLM-4的稳定性会相对更好，出现字段遗漏或者逻辑错误的情况也会比较少。不过即使是表现较优的模型，在复杂任务中仍然可能出现格式异常或

者字段解析错误的问题。

考虑到这些错误如果直接传递给后端的优化模块，就可能导致求解失败甚至引发程序异常，所以系统并没有完全依赖大模型的输出结果。在得到模型生成的参数信息之后，程序会进一步执行本地的规则校验，具体来说就是检查变量有没有超出取值范围、锁定特征有没有被误修改，以及调整变量的数量有没有超过限制。对于那些不符合要求的部分，则会通过规则进行修正或者恢复。依靠这种“大模型解析加上本地规则校验”的方式，就能在一定程度上降低解析错误对后续优化过程造成的影响，从而提高系统整体运行时的稳定性。

6.4 SHAP解释结果与决策一致性分析

在前文已经验证过预测模型精度的基础上，本节会进一步关注一个更实际的问题，那就是模型不仅要“预测准确”，还需要能解释出结果，并在复杂约束下给出合理的调整建议；因此这一部分主要从两个方面展开，一是SHAP解释结果在优化前后的变化情况，二是算法决策与关键特征之间的一致性状况。

6.4.1 优化前后解释结果的对比分析

验证一个优化方案是否有效，可以采用前后对比的方式。优化之前，程序会先调用SHAP方法对原始数据计算出各个特征的贡献度，计算之前会用K-Means压缩背景数据以减少耗时，不过这一步并不会影响到重要特征的排序。一开始的输出显示，程序能正确识别出那些会对预测结果产生较大影响的前几个关键特征。

当优化方案生成并且实际应用之后，程序会对优化后的特征参数重新计算一次SHAP值，然后将两次得到的重要度变化放在一起进行对比。从结果图上看，那些被算法改动过的关键变量，原先偏高的负向权重有一定程度的下降，部分特征甚至转成正向贡献。相比只给出一张优化前的特征贡献图，这种前后对比的方式更能说明问题，也就是优化算法确实抓住主要矛盾，并且达到预期的数值调整效果。

6.4.2 决策一致性的量化与约束影响分析

除关注预测结果本身，系统还会进一步分析另一个问题，那就是优化算法实际调整的那些变量，与SHAP识别出来的关键特征之间是否具有比较高的一致性。为量化这种关系，本文在程序里引入“决策对齐度(Align@K)”指标，并且分别在能耗数据集和信贷数据集上进行测试。

在实验过程中，系统会始终保持“最多只能修改K个变量”这一约束条件不变，仅仅对是否加入“变量锁定”这个限制进行区分。其中没有锁定条件表示算法可以自由选择需要调整的变量，而有锁定条件则表示一部分特征被固定，无法参与修改。

具体的量化对比结果如图6-1所示：

图 6-1 决策对齐度指标

在没有设置变量锁定条件的时候，算法在两个数据集上的对齐度分别达到77.1%和60.8%。这个结果说明，在没有额外限制的情况下，优化过程会优先选择那

2.8%(260)

些对结果影响较大的关键变量进行调整，这与SHAP所得到的特征重要性排序基本保持一致。

当加入“变量锁定”这一比较强的约束之后，两个数据集上的对齐度都出现比较明显的下降。造成这一现象的原因并不是算法的优化能力下降，而是实际的业务条件对优化空间造成限制。

在真实场景当中，一些贡献度比较高的核心变量往往会受到设备条件、业务规则或者人为要求的影响，无法直接修改，因此算法只能转而调整那些影响相对更小一些的变量，对齐度也会随之降低。

从这组实验结果可以看出，对齐度的变化实际上反映优化过程从“理论最优”向“实际可行”进行调整的过程。这也说明，当系统面对部分变量不可修改的限制条件时，仍然能够在约束范围内寻找可行方案，而不是停留在理想化的最优结果上。

6.5 受限条件下的优化效果分析

在实际业务调度的过程中，系统接收到的优化需求通常会带有比较明确的约束条件，例如要求一部分变量保持不变，或者限制只能调整少量参数。一旦这些约束被加入到优化过程中，算法所能搜索到的解空间就会明显缩小。基于这一情况，本节会通过对比实验，对系统在加入“基因修复机制”之后处理复杂约束条件时的求解能力进行验证。

6.5.1 实验设置与对照方案

本次测试分别在能耗数据集（8 个维度）和信贷数据集（20 个维度）这两个数据集上展开。针对每一个数据集，系统会先使用大模型模拟人类可能发出的自然语言指令，并随机生成100组带有双重限制的测试任务，第一重限制是修改数量不能超过某个值，第二重限制是指定某些特定特征不允许改变。在代码的分支对比上，一共设立下面两个实验组：一是基准算法，也就是原始的NSGA-II，它会保持原有的算法架构不变，遇到越界的数据时只进行惩罚分的扣减，不添加任何人为的干预。二是改进方法，也就是带有修复机制的那一组，它会采用本文所编写的约束前置校验模块以及基因强制回写模块。评价标准则采用“求解成功率”，也就是只要程序能在规定的迭代代数之内输出至少一个完全满足约束条件的方案，本次任务就会判定为成功。

6.5.2 消融实验结果与深度分析

从实验结果可以看出，在没有加入修复机制的情况下，原始的NSGA-II算法在8维的能耗数据集上取得62.1%的求解成功率。而当数据的维度提升到20维的信贷数据集之后，这个成功率进一步下降到53.5%。这一结果说明，随着特征维度的不断增加，算法在强约束条件下面获得可行解的难度也会随之提高，求解的稳定性会出现比较明显的下降。

通过分析实验过程可以发现，导致成功率下滑的主要原因在于，遗传算法交叉

与变异产生的解往往不符合约束条件。在严格限制变量修改数量的条件下，种群进化的过程中，会大量生成子代个体，这些个体很多不满足设下的约束条件。基准组仅对违规个体做扣分处理，降低其竞争力，但是并没有将其移出，导致这些不可行解个体仍然参与后续的进化过程，造成计算资源的浪费。随着进化次数增加，种群中不满足限制条件的个体逐渐累积，最终使程序难以稳定输出合法解。

实验结果如图6-2所示：

图 6-2 求解成功率指标

测试结果显示，在加入变量修复与回写机制后，两个数据集上的求解成功率都提升到100%。从具体执行过程来看，系统在个体适应度计算之前增加一层约束检查步骤。程序会对新生成个体的状态进行判断，一旦发现变量修改数量超过限制，或者修改被锁定的特征，就会对相应参数进行调整，使其恢复到满足约束的范围内。经过处理后的合法个体，才能继续进入后续计算过程，同时修正后的参数也会直接覆盖原有结果，以保证种群中的个体始终满足约束条件。

后续迭代过程可以发现，这些经过修复的个体能够正常参与种群进化，并逐渐提高可行解在种群中的比例，整个搜索过程也因此变得更加稳定。对比实验结果可以看出，原始NSGA-II在强约束场景下的问题，并不完全来自搜索能力本身，而是在进化过程中缺少对非法个体的有效处理机制。本文加入的修复逻辑，相当于在算法内部增加一步约束校正，使种群始终在合法解空间附近进行搜索，从而缓解复杂限制条件下容易出现的求解停滞问题。

6.6 结果分析与讨论

不同预测模型在各类数据场景中的表现存在一定差异。对于能耗数据这类维度较低、变量关系相对明确的数据，基于树结构的方法依然能够取得较好的效果。而在信贷数据这类高维且变量关系较复杂的任务中，引入注意力机制的模型，在特征关联建模方面表现出更强的能力。由实验结果可以看出，模型效果并不完全取决于结构是否复杂，更重要的是模型与数据特征之间是否能够较好匹配。

在可解释性分析这一部分中，系统除会对当前状态下的特征重要性进行计算之外，还会结合SHAP方法比较优化前后特征贡献所发生的变化。结果显示经过调整之后的关键变量，它们所对应的负向影响已经有比较明显的减弱。通过这种前后对比的方式，不仅能分析出优化过程中实际调整哪些重要的特征，也能让整个优化结果变得更容易让人理解。

为进一步分析决策结果与特征贡献之间的一致性，本文引入Align@K指标来作为量化的依据。在没有额外约束、可以自由调整变量的情况下，算法通常会优先修改那些贡献度比较高的核心特征。而一旦加入“部分变量不可调整”之类的限制条件之后，对齐度指标就会出现比较明显的下降。这一变化与实际业务场景会较为接近，原因是部分关键参数往往受到物理条件或者业务规则的限制，无法直接修改，所以优化过程就只能转而调整那些影响相对更小一些的变量。

针对受限条件下的优化实验可以发现，传统的进化算法在约束条件较强时，会比较难稳定地获得满足条件的可行解，其主要原因在于搜索过程中会产生大量违反约束的个体，这些无效解会占用掉较多的计算资源，同时也影响整体的搜索效率。在加入修复机制之后，那些不满足条件的个体会在接受评估之前被调整到合法的范围之内，并且继续参与后续的迭代过程，这样一来搜索过程就会变得更加稳定，求解的成功率也有比较明显的提升。这说明在实际应用场景中，仅仅依赖原有的优化算法往往很难满足复杂的约束需求，还需要结合具体的问题对算法做针对性的调整。

从整体流程来看，系统已经形成一套比较完整的处理过程，一是大语言模型负责把自然语言需求转换成结构化的约束，二是预测模型用来建立变量之间的映射关系，三是SHAP方法用于解释各个变量对结果造成的影响，四是优化算法则在满足约束条件的基础之上输出可行的方案。各个模块之间会相互配合，共同完成系统的整体功能。

6.7 本章小结

本章通过实验对系统各部分功能进行验证与分析。在预测模型部分，实验结果表明，Tabular Transformer在处理高维且结构较复杂的表格数据时，能够取得较好的拟合效果。

在自然语言解析部分，比较不同模型之后能发现，一旦结合规则校验，解析结果的稳定性就有一定程度的提升。

在可解释性分析方面，本文用SHAP方法对特征贡献做量化，并且结合优化前后的变化对模型行为做进一步的分析，同时又通过引入对齐度指标，说明约束条件会给优化结果带来哪些影响。

在受限优化实验中，引入修复机制之后，无效解的数量会明显减少，算法在复杂约束条件下的求解能力也得到提升。

总体来看，本章从预测效果、结果解释以及约束条件下的优化能力等几个方面，对系统的实际表现进行较为全面的分析与验证。

8.7总结与展望

AI特征值: 0.0%AI特征字符数 / 章节(部分)字符数: 0 / 1982

片段指标列表

序号	片段名称	字符数	AI特征
----	------	-----	------

原文内容

7 总结与展望

7.1 全文总结

随着工业生产和金融业务中数据规模的不断增加，单纯依靠人工经验来做决策会变得越来越困难。近年来，深度学习模型在预测任务中的表现虽然有不断提升，但在实际应用过程中还会存在一些比较明显的问题，第一，模型内部计算过程比较复杂，结果会缺少直观的解释；第二，真实业务环境中往往存在大量约束条件，仅有预测结果并不能直接支持实际决策。所以如何把预测、解释以及约束优化结合起来，就成为一个值得研究的问题。

围绕上述问题，本文设计并且实现一套面向多变量场景的预测与决策辅助系统。系统整体包含预测模型、可解释分析以及受限优化这三个大的部分，并且结合大语言模型实现自然语言约束的解析。论文的主要工作有下面几点：

首先，本研究构建面向表格数据的多变量预测模型。针对工业与金融场景中常见的高维表格数据，本文分别实现XGBoost、MLP以及Tabular Transformer等模型，并且做对比实验。其中的Tabular Transformer会通过注意力机制对不同特征之间的关系进行建模，所以在高维复杂数据上会有相对更稳定的表现。实验结果表明，该模型在信贷数据等复杂场景下具有比较好的拟合能力，也能为后续优化过程提供比较可靠的预测基础。

其次，本研究实现基于SHAP的可解释分析方法。为提高模型结果的可解释性，本文引入SHAP方法并用于分析特征贡献。与传统上只解释单次预测结果的那种方式不同，本文会对优化前后的特征贡献变化做一次对比，从而分析出变量调整会给结果带来哪些影响。同时再结合Align@K指标，本研究对优化方案与关键特征之间的一致程度做量化分析，这样就能让结果验证过程变得更加直观。

然后，本研究实现自然语言约束解析以及受限优化的流程。考虑到实际业务需求通常会以自然语言形式给出，本文会利用大语言模型解析用户的输入，并将其转换成结构化的约束条件。在此基础上，以NSGA-II为核心实现多目标优化。针对传统算法在强约束条件下容易产生大量不可行解的问题，本文引入基于拉马克思想的修复与回写机制，并对不满足约束条件的个体做出调整，这样能提高算法在复杂条件下的求解能力。

最后，本研究完成系统实验与综合验证。本文选取建筑能耗和金融信贷这两个不同领域的数据集来做实验分析，并且从预测性能、解释结果以及约束条件下的优化效果等多个方面做测试。实验结果表明，该系统能比较好地完成多变量预测、结果解释以及受限条件下的决策优化任务，这也验证整体方案的可行性。

总体来看，本文尝试把深度学习预测、可解释分析以及约束优化这三者结合到一起，从而形成一套相对完整的处理流程。虽然系统还存在一定局限，不过实验结果说明，该方法在复杂业务场景中会具有一定的应用价值。

7.2 研究展望

本文主要围绕多变量预测与受限优化问题做初步的研究，不过由于时间和实验条件有限，系统还会有一些可以进一步完善的地方，后续工作可以从下面几个方向继续展开：第一点，进一步考虑时间方面的因素，目前系统主要针对静态表格数据做建模，但在实际工业场景中许多变量会随时间不断发生变化，例如设备状态、能耗指标以及金融数据都具有比较明显的时间相关性，后续可以尝试引入时间序列模型，这样能增强系统对动态变化过程的建模能力。第二点，扩展系统的实际应用场景，目前实验主要基于公开数据集来完成，场景会相对有限，未来可以结合更多真实业务数据，对系统在工业调度、设备控制以及风险评估等场景中的表现做进一步的验证，并且根据具体需求针对性地改进模型结构和优化流程。总体而言，本文完成的工作更多属于一种探索性的尝试，随着深度学习、大语言模型以及智能优化方法的不断发展，预测、解释与决策这三者之间的结合方式还会有比较大的研究空间。后续若能将系统的稳定性与适应能力再进一步提升，这样它在实际场景中的应用价值有望得到进一步的提高。

致谢

本论文的完成离不开许多人的支持与帮助，在此谨向所有关心和帮助过我的老师、同学和家人表示衷心的感谢。

首先，我要向我的指导老师陈伟致以最诚挚的谢意。在论文选题、研究思路以及具体实现过程中，老师都给予了耐心细致的指导。从最初的方向确定到后续内容的反复修改，老师提出了许多具有针对性的宝贵建议，使我能够逐步理清思路并不断完善论文内容。

其次，感谢在学习和项目过程中给予我帮助的小伙伴们。无论是在技术问题的探讨中，还是在实验调试与结果分析时，大家毫无保留的交流与建议都对我的工作起到了积极的促进作用。

此外，我也要深深感谢我的家人一直以来的理解与支持。在论文写作的攻坚阶段，他们为我提供了稳定的生活环境和强大的精神鼓励，使我能够心无旁骛地专注完成本次研究工作。

最后，还要感谢大学期间所有任课教师的悉心教导。正是平日里扎实的专业积累，为本论文的顺利完成打下了坚实的基础。

说明:

- 1、支持中、英文内容检测；
- 2、 $\text{AI特征值} = \text{AI特征字符数} / \text{总字符数}$ ；
- 3、红色代表AI特征显著部分，计入AI特征字符数；
- 4、棕色代表AI特征疑似部分，未计入AI特征字符数；
- 5、检测结果仅供参考，最终判定是否存在学术不端行为时，需结合人工复核、机构审查以及具体学术政策的综合应用进行审慎判断。



关注微信公众号

知网AIGC检测服务

知网AIGC检测服务